

Advanced Optimization

Chapter 8 — Calculating Derivatives

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 8

Every algorithm so far eats derivatives. Where do they come from?

Line searches, trust regions, quasi-Newton, Newton-CG — all consume ∇f ; some consume $\nabla^2 f$ or its action $\nabla^2 f d$; Chapters 10–11 will consume Jacobians J .

Three sources

- ▶ **Finite differencing (§8.1)** — perturb and divide. Approximate; simple; what packages do when you supply nothing.
- ▶ **Automatic differentiation (§8.2)** — the function is code; code is a composition of elementary operations; apply the chain rule to the code. **Exact** (to roundoff).
- ▶ **Symbolic differentiation** — manipulate formulas (Mathematica, Maple). Not treated: it does not scale to functions defined by programs.

Why this chapter matters more now than in 2006

Reverse-mode AD is **backpropagation**. Every deep-learning framework is an AD engine with a linear-algebra library attached. This chapter is where you learn what those frameworks actually do — and what they cost.

Forward differences, and the Taylor bill for them

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + \epsilon e_i) - f(x)}{\epsilon}. \quad (8.1)$$

Full gradient: $n + 1$ evaluations of f .

From Taylor (2.6), with L a bound on $\|\nabla^2 f\|$:

$$\frac{\partial f}{\partial x_i}(x) = \frac{f(x + \epsilon e_i) - f(x)}{\epsilon} + \delta_\epsilon, \quad |\delta_\epsilon| \leq (L/2)\epsilon. \quad (8.4)$$

So take $\epsilon \rightarrow 0$? **No — the computer intervenes.**

Roundoff enters

Computed f carries relative error $\sim u$ (unit roundoff, $\approx 1.1 \times 10^{-16}$ in double). The difference quotient then has total error

$$\underbrace{(L/2)\epsilon}_{\text{truncation}} + \underbrace{2uL_f/\epsilon}_{\text{cancellation}}. \quad (8.5)$$

One term falls with ϵ , the other grows. The V-shaped error curve every numerical computing student should plot once.

Choosing ϵ , and the central alternative

Minimizing (8.5): $\epsilon^2 = 4L_f u/L$. For well-scaled problems (L_f/L moderate):

$$\boxed{\epsilon = \sqrt{u} \approx 10^{-8}} \quad \text{total error} \approx \sqrt{u} \approx 10^{-8}. \quad (8.6)$$

This is what optimization packages actually use.

Central difference (8.7)

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon} :$$

the $O(\epsilon)$ terms cancel; truncation is $O(\epsilon^2)$; optimal $\epsilon \approx u^{1/3}$, error $\approx u^{2/3} \approx 10^{-11}$. Price: $2n + 1$ evaluations.

	evals for ∇f	optimal ϵ	achievable error
forward (8.1)	$n + 1$	$u^{1/2} \approx 10^{-8}$	$\sim 10^{-8}$
central (8.7)	$2n + 1$	$u^{1/3} \approx 5 \times 10^{-6}$	$\sim 10^{-11}$

Half the significant digits are lost either way. Adequate for many line searches and quasi-Newton updates; marginal for tight tolerances — one reason to want AD.

Sparse Jacobians: pay per color, not per column

For $r : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with Jacobian J (8.9):

$$J(x)p \approx \frac{r(x + \epsilon p) - r(x)}{\epsilon} \quad (8.11)$$

— *one* extra evaluation for a Jacobian–vector product (this was (7.10) for $\nabla^2 f d$ with $r = \nabla f$). Full J column by column: $n + 1$ evaluations. **Sparsity does better.**

Estimate many columns per evaluation

If columns $i_1, \dots, i_\ell$ have *no row in common*, perturb them together: $p = \epsilon(e_{i_1} + \dots + e_{i_\ell})$. Each component of $r(x + p) - r(x)$ is touched by at most one of the columns, so all ℓ columns are recovered from one evaluation.

- Formalized by **graph coloring**: nodes = columns; arc when two columns share a row; one evaluation per color. A banded J with bandwidth w : about w colors, *independent of n* . The tridiagonal-type example (8.13) needs 3–4 evaluations of r , not $n + 1$.
- **Hessians**: apply the same to ∇f as a vector function — and *symmetry* halves the work again. The arrowhead Hessian (8.23) needs $n + 1$ evaluations by column-coloring, but only **2 perturbations** using symmetry: $p = \epsilon e_1$ gives row and column 1; $p = \epsilon(e_2 + \dots + e_n)$ gives all remaining diagonals at once. (General rule: adjacency-graph coloring where every 3-arc path uses ≥ 3 colors — Coleman–Moré.)

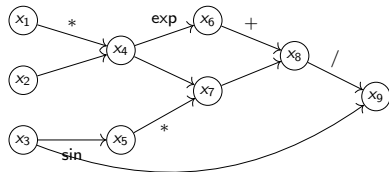
The premise: your code is already a composition

Any function, however complicated, is evaluated as a sequence of **elementary operations** on one or two arguments: $+$, $\times$, $/$, a^b , $\sin$, $\exp$, $\ln, \dots$. The chain rule (8.25) applies to each. AD applies it to *the program*:

$$f(x) = (x_1 x_2 \sin x_3 + e^{x_1 x_2}) / x_3 \quad (8.26)$$

$$\begin{aligned} x_4 &= x_1 x_2, & x_5 &= \sin x_3, \\ x_6 &= e^{x_4}, & x_7 &= x_4 x_5, \\ x_8 &= x_6 + x_7, & x_9 &= x_8 / x_3. \end{aligned} \quad (8.27)$$

x_1, x_2, x_3 : independent variables;
 $x_4, \dots, x_9$: intermediate; $x_9 = f$.



Computational graph (Figure 8.2)

The tool builds this graph itself (operator overloading or source transformation) — the user writes ordinary code. **Not** symbolic (no formulas grow), **not** numerical differencing (no ϵ , no cancellation): exact derivatives of the program, to roundoff.

Forward mode: carry a directional derivative through the graph

Fix a **seed vector** $p \in \mathbb{R}^n$. Alongside every variable x_i , propagate the scalar

$$D_p x_i = (\nabla x_i)^T p \quad (8.28)$$

(gradient w.r.t. the independent variables). Initialization: $D_p x_i = p_i$ for $i = 1, \dots, n$. Each elementary operation has a paired derivative operation, e.g.

$$x_7 = x_4 x_5 \Rightarrow D_p x_7 = x_5 D_p x_4 + x_4 D_p x_5; \quad z = \frac{w}{y} \Rightarrow D_p z = \frac{1}{y} D_p w - \frac{w}{y^2} D_p y. \quad (8.29-8.30)$$

At the output node: $D_p x_9 = \nabla f(x)^T p$. **One sweep = one directional derivative.**

- ▶ Full gradient: run n seeds $e_1, \dots, e_n$ (simultaneously) — cost up to $n \times$ the function, storage up to n extra scalars per variable (sparsity of early D 's helps).
- ▶ No graph storage needed: once a node's children are done, it can be overwritten.
- ▶ Implementations: operator overloading, or a precompiler emitting derivative code.

Forward mode is the right tool when you want a few directional derivatives — e.g. Jp for inexact Newton (Ch. 11), or $\nabla^2 f p$ (below). It is the wrong tool for a gradient with $n = 10^6$.

Reverse mode: one backward sweep for the whole gradient

Associate with each node an **adjoint** $\bar{x}_i$, accumulating $\partial f / \partial x_i$. Initialize all $\bar{x}_i = 0$ except the output: $\bar{x}_N = 1$.

The reverse sweep

Process nodes from output to inputs. When node j is *finalized* ($\bar{x}_j = \partial f / \partial x_j$), it pays each parent i :

$$\bar{x}_i += \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial x_i}. \quad (8.31\text{--}8.32)$$

The local partials $\partial x_j / \partial x_i$ (one per arc) were computed and stored during the *forward* evaluation of f . When all nodes are finalized, $(\bar{x}_1, \dots, \bar{x}_n) = \nabla f(x)$.

Example (8.26) at $x = (1, 2, \pi/2)$: node 9 finalizes first and pays nodes 8 and 3 ($\bar{x}_8 += \frac{1}{\pi/2}$,

$\bar{x}_3 += -\frac{2+e^2}{(\pi/2)^2}$); then 8 pays 6 and 7; ...; three sweep steps later

$\nabla f = ((4 + 4e^2)/\pi, (2 + 2e^2)/\pi, (-8 - 4e^2)/\pi^2)$. Note $\bar{x}_3$ waits for *two* contributions (children 5 and 9) — adjoints add up.

Numerical values flow, not formulas. Computation runs children $\rightarrow$ parents — the exact reverse of evaluation.

The cheap gradient principle — and its price

Cost of reverse mode, scalar $f : \mathbb{R}^n \rightarrow \mathbb{R}$

$$\text{cost}(f \text{ and } \nabla f) \leq \sim 5 \times \text{cost}(f), \quad \text{independent of } n.$$

(Each elementary op spawns at most a handful of adjoint ops; the division node costs about 5.)

Compare: forward mode or finite differences pay $\sim n \times$. For $n = 10^6$, that is the difference between one gradient and a million function evaluations.

This asymmetry runs the modern world

Reverse-mode AD *is* backpropagation. Training a network with 10^9 parameters is feasible only because the gradient costs a small constant times the forward pass. The 1960s–80s AD literature and the neural-network literature discovered it independently.

The price: memory

The reverse sweep needs the graph (values + arc partials from the forward pass); one second of evaluation at 100 Mflop $\sim$ 2 GB. Remedy: **checkpointing** — store snapshots, recompute segments during the sweep. DL frameworks’ “gradient checkpointing” is exactly this trade of arithmetic for memory.

Vector functions: pick the mode by the shape of J

$r : \mathbb{R}^n \rightarrow \mathbb{R}^m$, Jacobian $J \in \mathbb{R}^{m \times n}$.

Forward, seed $p \in \mathbb{R}^n$

One sweep yields Jp .

Full J : n seeds (or # colors of J 's columns).

Natural when n small or only Jp needed
(inexact Newton, Ch. 11: one sweep per product).

Reverse, seed $q \in \mathbb{R}^m$

Apply reverse mode to $r(x)^T q$: one sweep yields $J^T q$.

Full J : m seeds (or # colors w.r.t. J^T), $\leq 5 \times$ each.

Natural when $m \ll n$ — a scalar loss over many parameters is the $m=1$ case.

Modes can be combined — some columns by forward seeds, remaining rows by reverse seeds. (In framework language: `jvp` is a forward sweep, `vjp` is a reverse sweep.)

Hessians by AD

- ▶ **Forward with two seeds:** propagate $D_{pq}x_i = p^T(\nabla^2 x_i)q$ (8.36) alongside $D_p x_i$; the output node yields $p^T \nabla^2 f(x)q$.
- ▶ **Univariate trick** (8.39–8.40): with $\psi(t) = f(x + tp)$, $D_p f = \psi'(0)$ and $D_{pp}f = \psi''(0)$; the interpolation identity

$$[\nabla^2 f]_{ij} = \frac{1}{2} [(e_i + e_j)^T \nabla^2 f(e_i + e_j) - e_i^T \nabla^2 f e_i - e_j^T \nabla^2 f e_j]$$

recovers sparse Hessians from pure second directional derivatives — no cross terms to propagate.

- ▶ **Reverse over forward:** forward-propagate $D_q x_i$ (computing $\nabla f^T q$), then reverse-sweep that scalar $\Rightarrow \nabla^2 f(x)q$ at $\sim 12\times$ the cost of f , *independent of n* — the exact replacement for the finite difference (7.10) in Hessian-free Newton-CG. Full sparse Hessian: $\sim 12N_c$ with coloring.

What AD cannot do (current limitations)

- ▶ **AD differentiates the program, not the mathematics.** If f is computed through an inner solver (a PDE discretization, an iterative loop), the computed $\hat{f} = f + \tau$ carries truncation error τ ; $|\tau|$ small does **not** make τ' small. The AD gradient is the exact gradient of $\hat{f}$ — which may be a poor gradient of f . (Finite differences suffer identically.)
- ▶ **Branches.** Code `if (x == 1.0) f = 0.0 else f = x-1.0` computes $f(x) = x - 1$ everywhere, but AD at $x = 1$ returns $f'(1) = 0$. Piecewise implementations of special functions cause milder versions of the same.
- ▶ **Nonsmooth primitives** ($|\cdot|$, `max`, `ReLU`...): AD returns a one-sided value at kinks. Fine almost everywhere; know that it happens.

The lesson

AD is exact for what the code does. Whether the code does what the mathematics says is *your* problem — and it is exactly the issue when differentiating through simulations and solvers. Which brings us to...

Beyond the book: adjoints of solvers, in one slide

Often $f(x) = \varphi(u(x))$ where $u(x)$ is defined *implicitly*: it solves $c(u, x) = 0$ (a power flow, a PDE, an inner optimization's KKT conditions).

Two ways to differentiate

- ▶ **Unroll**: apply reverse-mode AD through all solver iterations. Correct for the *computed* u ; memory grows with iteration count; the gradient inherits solver noise (τ' from the last slide).
- ▶ **Implicit / adjoint**: differentiate the *condition* $c(u, x) = 0$ instead:

$$\frac{df}{dx} = -\lambda^T \frac{\partial c}{\partial x}, \quad \text{where} \quad \left(\frac{\partial c}{\partial u}\right)^T \lambda = \left(\frac{\partial \varphi}{\partial u}\right)^T$$

— **one linear solve with the transposed Jacobian**, no matter how many iterations the forward solver took.
Cost: one factorization (often reusable from the solver itself).

- ▶ This is the **adjoint method** of PDE-constrained optimization — the implicit function theorem applied to computation. Differentiable layers wrap solvers this way: backprop through a Newton power flow or a KKT system uses the adjoint solve, not unrolled iterations.
- ▶ Reverse-mode AD and the adjoint method are one idea at two granularities: elementary operations vs. an entire equation solve as the unit.

Chapter 8 in one table

Want	Method	Cost (vs. one f or r)	Exact?
∇f , quick and dirty	forward FD (8.1), $\epsilon = \sqrt{u}$	$n+1$	$\sim 10^{-8}$
∇f , more digits	central FD (8.7), $\epsilon = u^{1/3}$	$2n+1$	$\sim 10^{-11}$
∇f , n large	reverse AD	≤ 5	yes
Jp or $\nabla^2 f d$	FD (8.11)/(7.10)	1 gradient/residual	$O(\epsilon)$
Jp	forward AD	small const	yes
$J^T q$	reverse AD	≤ 5	yes
full sparse J or $\nabla^2 f$	coloring + FD or AD	#colors	per method
$\nabla^2 f q$	reverse over forward	~ 12	yes
df/dx through a solver	adjoint / implicit	+1 transposed solve	yes (of the math)

Three ideas to carry forward

Finite differencing is a bias–variance trade: truncation vs. cancellation, optimum at $\sqrt{u}$. Reverse mode makes gradients cost $O(1)$ functions — the fact the modern world runs on. AD is exact about the program; the adjoint method is exact about the mathematics.

Homework

Derivations

- 1 Minimize (8.5) over ϵ to derive (8.6); repeat the analysis for the central difference to obtain $\epsilon \sim u^{1/3}$ and error $\sim u^{2/3}$.
- 2 For the arrowhead Hessian (8.23), write out why the two perturbations ϵe_1 and $\epsilon(e_2 + \dots + e_n)$ recover every nonzero entry.
- 3 Write the forward-mode propagation rules for $z = wy$, $z = \sin w$, $z = e^w$, $z = w/y$, and the corresponding reverse-mode adjoint updates for each.
- 4 Carry out the full reverse sweep for (8.26) at $x = (1, 2, \pi/2)$, node by node, and verify the gradient given in class.
- 5 Show that computing f and $\nabla f^T q$ jointly by forward mode, then reverse-sweeping, yields $\nabla^2 f(x)q$; count the cost factor.

Implementation

6. *The V-plot.* For Rosenbrock at $x = (-1.2, 1)$, compute forward and central FD approximations of $\partial f / \partial x_1$ for $\epsilon = 10^{-1}, \dots, 10^{-15}$; plot absolute error vs. ϵ on log-log axes. Mark $\sqrt{u}$ and $u^{1/3}$.
7. *Gradient checker.* Write the utility you will use all semester: compare an analytic (or AD) gradient against central differences, report the max relative error.
8. In your AD framework of choice, compute $\nabla^2 f(x)q$ three ways — finite difference of gradients (7.10), forward-over-reverse AD, full Hessian times q — for extended Rosenbrock, $n = 10^4$. Compare time and accuracy.

Next: Ch. 10–11 — *least squares and nonlinear equations: where Jacobians go to work.*